

Getting Started with Python



Course Instructor

Joseph Yeates

About Joseph

Joseph is passionate about driving better decision-making by creating, curating, and communicating insights from data.



Based in
Vancouver,
B.C.

Experience

A data enthusiast with a background in Retail Banking, he is an advocate for making advanced analytics more accessible to a wider audience by integrating them into business intelligence solutions.

Joseph helps organize the Vancouver Power BI & Modern Excel User Group. An active speaker, he has presented sessions on combining data science with business intelligence at conferences in the Pacific Northwest and across North America.

Getting Started with Python

Learning Objectives

01

Install Anaconda Navigator and generate a Jupyter Notebook.

02

Create and define variables and data types.

03

Identify common types of data structures and explain the differences between them.

04

Compare and analyze data with operators and functions.

05

Build more complex logic with conditional statements and for loops.

Getting Started with Python

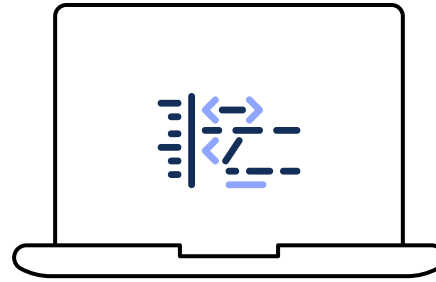
In this course, we will look at **foundational Python concepts** that will allow us to **generate more complex code**.



Variables &
Data Types



Operators &
Functions



Data Structures



Conditional
Statements &
For Loops

By the end of this course, we will be ready to start using Python to analyze our data and generate insights.

Variables & Data Types

In this module, we will be looking at **variables**, **data types**, and **how to convert & combine data types**.

Variables	Data Types	Convert & Combine Data Types															
Assigning: <code>x = 7</code>	<table><tr><th>Data Type</th><th>Values</th><th>Example</th></tr><tr><td>Integer</td><td>Whole numbers</td><td>1, 2, 3</td></tr><tr><td>Float</td><td>Decimal numbers</td><td>1.4, 2.78, 3.0091</td></tr><tr><td>Boolean</td><td>True or False</td><td>True, False</td></tr><tr><td>String</td><td>Text</td><td>'a', 'Hello', '3ab'</td></tr></table>	Data Type	Values	Example	Integer	Whole numbers	1, 2, 3	Float	Decimal numbers	1.4, 2.78, 3.0091	Boolean	True or False	True, False	String	Text	'a', 'Hello', '3ab'	<code>int()</code> - converts input to integer
Data Type	Values	Example															
Integer	Whole numbers	1, 2, 3															
Float	Decimal numbers	1.4, 2.78, 3.0091															
Boolean	True or False	True, False															
String	Text	'a', 'Hello', '3ab'															
Printing: <code>print()</code>		<code>float()</code> - converts input to float															
Reassigning		<code>string()</code> - converts input to string															

Variables & Data Types are a foundational piece for writing code in Python.

Data Structures

In this module, we will see how data structures can be used to **group** our **data and variables together**.

Data Structures

Data Structure	Syntax	Mutable?	Duplicates?	Example
Lists	[]	Yes	Yes	List_a = ['a','b','c'] List_b = [1,2,3] List_c = [1, 'a', True, 2.56]
Tuples	()	No	Yes	tuple_a = ('a','b','c') tuple_b = (1,2,3) tuple_c = (1, 'a', True, 2.56)
Dictionaries	{ }	Yes	No	dict_a = {'first_name': 'Frank', 'last_name': 'Park', 'age': 20}

Data Structures

In this module, we will see how data structures can be used to **group** our **data and variables together**.

Manipulating Lists

Syntax

`list.sort()`

`list.sort(reverse=True)`

`list.append()`

`list[1] = 'New Value'`

`del list[1]`

Definition

sort in ascending order

sort in descending order

add new items to a list

replace items in a list

delete items in a list

Data structures are an important concept because we use them to set the framework of larger projects and analysis.

Operators & Functions

In this module, we will examine **operators** and **functions** – which we can use to **interact with our data**.

Operators

Math Operators

`+, -, *, /`

`**` - exponent

`%` - modulo

Comparison Operators

`==`

`!=`

`<, >`

Logical Operators

`and, or`

`!=` - not

`==` - is

Operators & Functions

In this module, we will examine **operators** and **functions** – which we can use to **interact with our data**.

Functions

Exploratory Data Analysis

`type()`

`max()`

`min()`

`len()`

Data Transformation

`abs()`

`sum()`

`round()`

`pow()`

Operators and **functions** are crucial because they can save us time - allowing us to use logic that is already defined.

Conditional Statements & For Loops

In this module, we will start to combine some of the earlier concepts to build more complex logic into our code.



Conditional statements and **for loops** standardize logic and optimize code can significantly enhance your Python programming skills, regardless of the specific application.

Getting Started with Python

In this course, we reviewed foundational concepts that will allow us to generate more complex Python code.



Create and define **variables**

1 2 3

Set the **data type** of our variables and data



Combine our data into different **data structures**



Use **operators** and **functions** to run calculations and produce output



Build more complex logic by leveraging **conditional statements** and **for loops**

We are now ready to take the next step in our learning journey, using Python to analyze our data and generate insights!